Capstone Project 1

Enterprise Microservices Platform on AWS (ECS + Aurora + Redis + API Gateway + Okta)

Industry Scenario

A **FinTech company** migrating from on-prem monolithic architecture to cloud-native microservices.

They need:

- Containerized services
- Secure authentication via Okta
- High availability
- Caching layer
- Secret management
- Production-grade DNS + Load balancing

Problem Statement

Build a secure, scalable microservices architecture where:

- APIs are containerized and deployed on ECS Fargate
- Authentication via Okta OIDC
- Aurora PostgreSQL for transactional data
- Redis (ElastiCache) for caching
- Secrets stored in SSM Parameter Store
- Route53 custom domain
- Encrypted data using KMS
- ALB for load balancing

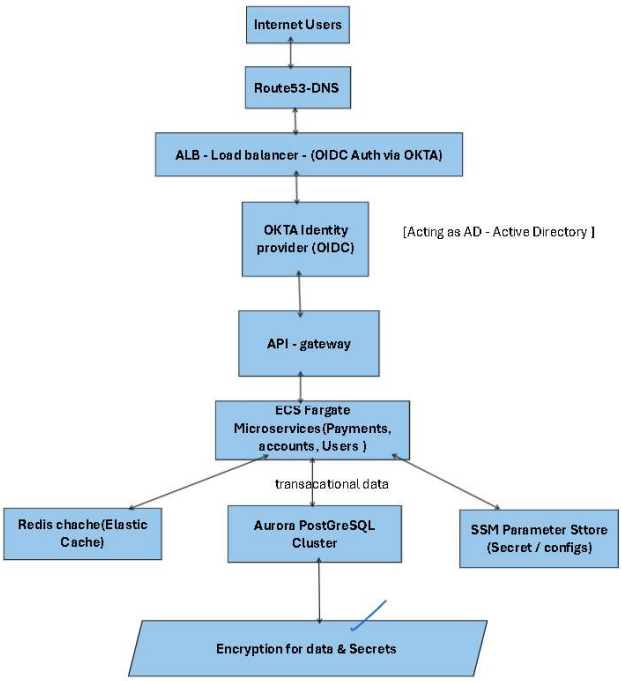
Use Cases

1. Customer login via Okta
2. Payment API calls
3. High-read caching from Redis
4. Secure DB access
5. Blue-green deployment ready infra

Architecture Components

- VPC (Multi-AZ)
- ECS Fargate
- ECR
- ALB
- Aurora PostgreSQL
- ElastiCache Redis
- KMS
- SSM Parameter Store
- Route53
- Okta (OIDC)

Enterprise Microservices Platform on AWS - Architecture Diagram:



Request Flow (Step-by-Step)

1 User Access

User accesses application via custom domain: https://api.slkbangalore1.com
Managed by Amazon Route 53.

2 Load Balancer Authentication:

Traffic goes to: AWS Application Load Balancer
ALB performs OIDC authentication with Okta.

User → ALB → Okta → Token → ALB

3 API Routing
After authentication:
ALB → API Gateway → ECS Fargate
Microservices run in containers on AWS Fargate.
Example services: Customer Service Payment Service Transaction Service, CART etc

4 Cache Layer:
High-read requests use: Amazon ElastiCache for Redis
Example:

Check Redis Cache
|
Cache Hit → Return data
Cache Miss → Query Database

5 Database Layer
Transactional data stored in: Amazon Aurora PostgreSQL
Features: Multi-AZ Automatic failover, High performance

6 Secret Management
Sensitive data stored in: AWS Systems Manager Parameter Store
/prod/db/password
/prod/api/secret

Encrypted using: AWS Key Management Service

Infrastructure Layer (Multi-AZ VPC):

```
VPC
├── Public Subnets
│   ├── Application Load Balancer
│
└── Private Subnets
    ├── ECS Fargate Tasks
    ├── Aurora PostgreSQL
    ├── Redis Cluster
```

Solution Steps

1. Create VPC with public & private subnets
2. Create KMS key
3. Store DB credentials in SSM
4. Deploy Aurora cluster
5. Deploy Redis cluster
6. Create ECR repo
7. Deploy ECS service
8. Configure ALB
9. Configure Route53
10. Integrate Okta OIDC in ALB

Terraform Code (Core Snippets)

1 Provider & Backend

```
provider "aws" {
  region = "us-east-1"
}

terraform {
  backend "s3" {
    bucket = "fintech-tf-state"
    key    = "prod/ecs-platform.tfstate"
    region = "us-east-1"
  }
}
```

---

2 KMS Key

```
resource "aws_kms_key" "platform_key" {
  description             = "KMS key for platform encryption"
  deletion_window_in_days = 7
  enable_key_rotation     = true
}
```

---

3 SSM Parameter Store

```
resource "aws_ssm_parameter" "db_password" {
  name  = "/prod/db/password"
  type  = "SecureString"
  value = "SuperSecret123"
  key_id = aws_kms_key.platform_key.id
}
```

---

4 Aurora PostgreSQL

```
resource "aws_rds_cluster" "aurora" {
  engine         = "aurora-postgresql"
  master_username = "admin"
  master_password = aws_ssm_parameter.db_password.value
  kms_key_id     = aws_kms_key.platform_key.arn
  skip_final_snapshot = true
}
```

---

5 ElastiCache Redis

```
resource "aws_elasticache_cluster" "redis" {
  cluster_id           = "redis-cache"
  engine               = "redis"
  node_type            = "cache.t3.micro"
  num_cache_nodes      = 1
}
```

---

6 ECR Repository

```
resource "aws_ecr_repository" "app_repo" {
  name = "payment-service"
}
```

---

7 ECS Task Definition

```
resource "aws_ecs_task_definition" "app_task" {
  family                   = "payment-service"
  network_mode             = "awsvpc"
  requires_compatibilities = ["FARGATE"]
  cpu                      = 512
  memory                   = 1024

  container_definitions = jsonencode([
    {
      name  = "app"
      image = aws_ecr_repository.app_repo.repository_url
      portMappings = [{
        containerPort = 8080
      }]
    }
  ])
}
```

---

8 ALB + Listener

```
resource "aws_lb" "alb" {
  name               = "platform-alb"
  load_balancer_type = "application"
  subnets            = var.public_subnets
}
```

---

9 Route53

```
resource "aws_route53_record" "app_dns" {
  zone_id = var.zone_id
  name    = "api.company.com"
  type    = "A"

  alias {
    name                   = aws_lb.alb.dns_name
    zone_id                = aws_lb.alb.zone_id
    evaluate_target_health = true
  }
}
```